VOLUME 3 OF 12

IT STUDY HUB

COMPLETE JAVA PROGRAMMING SERIES

MODULE 3

CONTROL FLOW & METHODS

Authored by:

Mithila Rani Saha & Dithsa Dutta

www.itstudyhub.dpdns.org

Welcome to Module 3

About This Module

Welcome to Volume 3 of the IT Study Hub Complete Java Programming Series. In Module 1, you learned the anatomy of a program. In Module 2, you learned how to store and calculate data. However, until now, every program you have written has executed sequentially—strictly from top to bottom, one line at a time.

Real intelligence requires decision-making. Programs must be able to adapt to different scenarios, skip unnecessary steps, and repeat tasks without you typing the same code thousands of times. This module introduces the "brain" of programming: **Control Flow** and reusable **Methods**.

Learning Outcomes

- Master decision-making structures using `if`, `if-else`, `else-if`, and `switch` statements.
- Automate repetitive tasks using `for`, `while`, and `do-while` loops.
- Control loop execution precisely using `break` and `continue` jump statements.
- Modularize your code by declaring and calling custom Methods.
- Understand the mechanics of Parameters, Arguments, and Return Types.
- Grasp advanced concepts like Method Overloading, Static Methods, Recursion, and Variable Scope.

Module Coverage

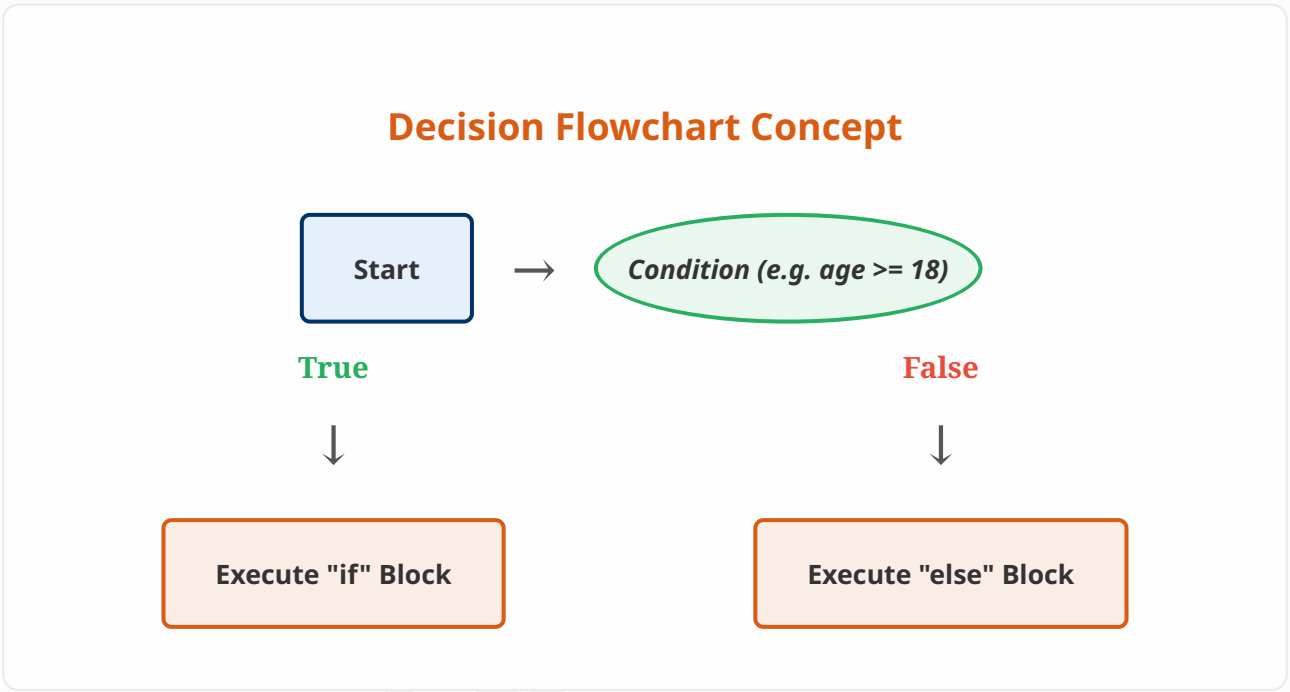
We start with Conditional branching logic, transitioning into iterative Loops. We then break out of the `main()` method box by writing our own custom functions. Finally, we combine all these elements in a comprehensive "Menu-Driven

Calculator" hands-on project. As always, this manual concludes with an extensive 45-question assessment and interview preparation section.

IT STUDY HUB

1. Conditional Statements (if, if-else, else-if)

Control flow statements change the order in which code is executed. The most fundamental type is conditional logic: executing a block of code *only if* a specific condition is met.



Concept: The basic 'if' and 'if-else' statements

> **Simple Explanation:**
> It's like a bouncer at a club. 'IF you have an ID, you may enter. ELSE, you must leave.'

Syntax / Structure:

```
if (condition) {
    // runs if condition is true
} else {
    // runs if condition is false
}
```

Example Code:

```
import java.util.Scanner;

public class EvenOddChecker {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter a number: ");
        int number = scanner.nextInt();

        // Check if the remainder is 0 when divided by 2
        if (number % 2 == 0) {
            System.out.println(number + " is an EVEN number.");
        } else {
            System.out.println(number + " is an ODD number.");
        }
    }
}
```

Expected Output:

```
Enter a number: 14
14 is an EVEN number.
```

Code Explanation:

The `if` statement checks `number % 2 == 0`. If the user inputs 14, the remainder is 0 (true), so the first block runs. If they input 15, the remainder is 1 (false), so Java skips the first block and executes the `else` block.

Real-world Example:
An ATM program: `if (requestedAmount <= accountBalance) { dispenseCash(); } else { showInsufficientFunds(); }`.

Common Mistakes:

- Putting a semicolon immediately after the if condition: `if (age > 18); { ... }`. This breaks the logic completely!

Best Practices:

- Always use curly braces `{}` even if there is only one line of code inside the block. It prevents errors if you add lines later.

Practice Task:
Write an if-else statement that checks if a user's password is exactly 'secret123'. If true, print 'Access Granted', otherwise print 'Access Denied'.

IT STUDY HUB

Concept: The 'else-if' ladder and Nested Conditions

> **Simple Explanation:**
> Instead of just yes/no, you have multiple options. 'If score > 90, grade A. Else if score > 80, grade B. Else, grade F.' Nested conditions are simply an `if` inside another `if`.

Syntax / Structure:

```
if (condition1) {
    // code
} else if (condition2) {
    // code
} else {
    // fallback code
}
```

Example Code:

```
public class LargestNumber {
    public static void main(String[] args) {
        int a = 15;
        int b = 25;
        int c = 10;

        if (a >= b && a >= c) {
            System.out.println("A is the largest");
        } else if (b >= a && b >= c) {
            System.out.println("B is the largest");
        } else {
            System.out.println("C is the largest");
        }
    }
}
```

Expected Output:

```
B is the largest
```

Code Explanation:
Java evaluates the first `if`. `15 >= 25` is false, so it skips to the `else if`. `25 >= 15 && 25 >= 10` is true, so it prints 'B is the largest'. It then skips the final `else` completely.

Real-world Example:
Calculating tax brackets based on exact income ranges.

Common Mistakes:

- Forgetting that once a true condition is found, the rest of the else-if ladder is entirely skipped. Order matters!

Best Practices:

- Put the most specific or most likely condition at the very top of the ladder to improve performance.

Practice Task:
Write an else-if ladder that takes an integer representing a temperature and prints 'Freezing', 'Warm', or 'Hot'.

IT STUDY HUB

2. The switch Statement

IT STUDY HUB

Concept: The 'switch' statement

> **Simple Explanation:**
> Imagine a multi-button control panel. Instead of asking 'Is it button 1? Is it button 2? Is it button 3?', a switch statement directly connects to the exact button pressed. We use the `break` keyword to stop the code from 'falling through' to the next button.

Syntax / Structure:

```
switch(variable) {
    case value1:
        // code
        break;
    case value2:
        // code
        break;
    default:
        // fallback code
}
```

Example Code:

```
public class DayOfWeek {
    public static void main(String[] args) {
        int dayCode = 3;
        String dayName;

        switch (dayCode) {
            case 1: dayName = "Monday"; break;
            case 2: dayName = "Tuesday"; break;
            case 3: dayName = "Wednesday"; break; // Matches here!
            case 4: dayName = "Thursday"; break;
            case 5: dayName = "Friday"; break;
            default: dayName = "Weekend"; // If 6, 7 or invalid
        }

        System.out.println("The day is: " + dayName);
    }
}
```

Expected Output:

```
The day is: Wednesday
```

Code Explanation:
Java looks at `dayCode`. It jumps directly to `case 3`. It assigns 'Wednesday' to the variable, and the `break` command tells it to exit the switch block immediately. If no case matches, the `default` block runs.

> ***Real-world Example:***
> *Menu systems in console applications (Press 1 to Start, 2 for Options, 3 to Exit).*

> **Common Mistakes:**
> - Forgetting the `break` keyword! If you forget it, Java will execute every single case below the matching one until it hits a break or the end of the block (this is called 'Fallthrough').

> **Best Practices:**
> - Always include a `default` case to handle unexpected or invalid input.

Practice Task:
Create a switch statement that takes a char 'A', 'B', or 'C' representing grades, and prints 'Excellent', 'Good', or 'Average'.

---

if-else vs. switch Comparison

Feature	if-else ladder	switch statement
Condition Type	Evaluates complex boolean conditions (e.g., >, <, &&).	Evaluates exact matching values only (equality).
Data Types Supported	Any expression that returns boolean.	byte, short, char, int, String, and Enums. (No float/double).
Readability		

	Can become messy if checking 10+ conditions.	Very clean and organized for menu-style logic.
Speed	Checks every condition from top to bottom.	Jumps directly to the correct case (slightly faster).

IT STUDY HUB

3. Looping Constructs (for, while, do-while)

Loops allow you to execute a block of code repeatedly as long as a condition remains true. Without loops, if you wanted to print "Hello" 1000 times, you would need 1000 lines of code.

IT STUDY HUB

Concept: The 'for' Loop

> **Simple Explanation:**
> A `for` loop manages the three crucial parts of looping in one neat line: 1. Start here. 2. Stop when this is false. 3. Take this step size.

Syntax / Structure:

```
for (initialization; condition; increment/decrement) {
    // code to run repeatedly
}
```

Example Code:

```
public class MultiplicationTable {
    public static void main(String[] args) {
        int tableNumber = 5;

        System.out.println("Table of 5:");
        // Loop runs from i = 1 to i = 5
        for (int i = 1; i <= 5; i++) {
            int result = tableNumber * i;
            System.out.println(tableNumber + " x " + i + " = " + result);
        }
    }
}
```

Expected Output:

```
Table of 5:
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
```

Code Explanation:

Execution Flow:

1. Initialize: `i = 1`.
2. Condition: `1 <= 5` (True).
3. Execute body (prints `5 x 1 = 5`).

4. Increment: `i++` makes `i` become 2.
5. Repeat condition check until `i` is 6, at which point the loop terminates.

Real-world Example:
Processing all elements in a database table or array sequentially.

Common Mistakes:

- Using commas instead of semicolons in the `for` loop declaration: `for(int i=0, i<10, i++)` (This is wrong).
- Causing an Infinite Loop by getting the condition wrong (e.g., `for (int i=1; i>0; i++)`). It will run forever until memory crashes!

Best Practices:

- Keep the loop variable (usually `i`, `j`, or `k`) strictly contained inside the loop declaration.

Practice Task:
Write a for loop that counts BACKWARDS from 10 down to 1.

Concept: The 'while' Loop

> **Simple Explanation:**
> It's like driving a car. You don't say 'I will steer 50 times.' You say, 'WHILE I am not at the destination, keep steering.' The condition is checked BEFORE entering the loop.

Syntax / Structure:

```
while (condition) {
    // code to run
    // remember to update the condition inside!
}
```

Example Code:

```
public class SimpleCounter {
    public static void main(String[] args) {
        int counter = 1; // Initialization is outside

        while (counter <= 3) {
            System.out.println("Count is: " + counter);
            counter++;   // Increment is inside! Crucial!
        }
    }
}
```

Expected Output:

```
Count is: 1
Count is: 2
Count is: 3
```

Code Explanation:

The `while` loop behaves identically to the `for` loop here, but the structural difference is that initialization happens before the loop, and the increment must be manually written inside the loop block.

> ***Real-world Example:***
> *Reading a file: `while (fileHasMoreLines()) { readLine(); }`*

Common Mistakes:

- Forgetting to update the variable inside the loop block, resulting in an infinite loop.

Best Practices:

- Use a `while` loop when the number of iterations depends on user input or external factors (like network data streams).

Practice Task:
Write a while loop that starts an integer at 2 and repeatedly multiplies it by 2 until it exceeds 100.

IT STUDY HUB

Concept: The 'do-while' Loop

> **Simple Explanation:**
> In a `while` loop, you ask for permission before entering. In a `do-while` loop, you barge in, do the action once, and THEN ask for permission to do it again.

Syntax / Structure:

```
do {
    // code to run at least once
} while (condition); // Notice the semicolon!
```

Example Code:

```
import java.util.Scanner;

public class GuessingLogic {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int choice;

        do {
            System.out.println("Menu: Enter 0 to exit, 1 to continue.");
            choice = scanner.nextInt();
        } while (choice != 0);

        System.out.println("Exited system.");
    }
}
```

Expected Output:

```
Menu: Enter 0 to exit, 1 to continue.
1
Menu: Enter 0 to exit, 1 to continue.
0
Exited system.
```

Code Explanation:
Even if `choice` was magically initialized to 0 beforehand, the menu prints *first*, gets the input, and only then checks `choice != 0`. This makes it perfect for menu systems.

Real-world Example:
Creating console-based interactive menus where you must display the menu options to the user before checking if they want to exit.

Common Mistakes:

- Forgetting the semicolon `;` at the very end of the `while(condition);` statement.

Best Practices:

- Always prefer `do-while` over `while` specifically for building user-driven menu interfaces.

Practice Task:
Write a do-while loop that prompts a user for a password until they type '1234'.

Loop Comparison Table

Loop Type	Best Used When...	Condition Check	Minimum Executions
for	The exact number of iterations is known in advance.	Pre-test (Before entry)	0
while	Iterations depend on a condition, might not run at all.	Pre-test (Before entry)	0
do-while	The code must execute at least once (like a menu).	Post-test (After entry)	1

Concept: Nested Loops

> **Simple Explanation:**
> Think of a digital clock. The 'minutes' digit loops from 0 to 59. But the 'hours' digit only loops once every time the 'minutes' digit finishes its entire 60-loop cycle. The inner loop must complete fully for every single step of the outer loop.

Syntax / Structure:

```
for (int outer = 1; outer <= 3; outer++) {
    for (int inner = 1; inner <= 3; inner++) {
        // code
    }
}
```

Example Code:

```
public class NumberPattern {
    public static void main(String[] args) {
        // Outer loop handles the rows
        for (int i = 1; i <= 3; i++) {
            // Inner loop handles columns
            for (int j = 1; j <= i; j++) {
                System.out.print(j + " "); // Use print, not println
            }
            System.out.println(); // Move to new line after row finishes
        }
    }
}
```

Expected Output:

```
1
1 2
1 2 3
```

Code Explanation:

Row 1 (`i=1`): Inner loop runs once (`j=1`), prints '1'. Row finishes.
Row 2 (`i=2`): Inner loop runs twice (`j=1` then `j=2`), prints '1 2'. Row finishes.
Row 3 (`i=3`): Inner loop runs thrice, prints '1 2 3'.

Real-world Example:
Generating graphics, working with 2D matrices/grids (like a chessboard).

Common Mistakes:

- Using the same counter variable (e.g., `i`) for both the inner and outer loops. They will clash and cause infinite loops or instant crashes.

Best Practices:

- Use standard conventions: `i` for outer loop, `j` for first inner loop, `k` for second inner loop.

Practice Task:
Create a nested loop to print a 3x3 square of asterisks (*).

IT STUDY HUB

4. Jump Statements (break, continue, return)

Sometimes you need to interrupt the normal flow of a loop before the condition naturally becomes false.

IT STUDY HUB

Concept: The 'break' and 'continue' keywords

> **Simple Explanation:**
> `break` is a fire alarm: drop everything and leave the building entirely. `continue` is a speed bump: slow down, skip this exact moment, but keep driving the rest of the route.

Syntax / Structure:

```
if (condition) break;
if (condition) continue;
```

Example Code:

```
public class JumpStatements {
    public static void main(String[] args) {

        System.out.println("--- BREAK EXAMPLE ---");
        for (int i = 1; i <= 5; i++) {
            if (i == 3) break; // Stops loop at 3
            System.out.println("Break loop: " + i);
        }

        System.out.println("--- CONTINUE EXAMPLE ---");
        for (int i = 1; i <= 5; i++) {
            if (i == 3) continue; // Skips 3, goes to 4
            System.out.println("Continue loop: " + i);
        }
    }
}
```

Expected Output:

```
--- BREAK EXAMPLE ---
Break loop: 1
Break loop: 2
--- CONTINUE EXAMPLE ---
Continue loop: 1
Continue loop: 2
Continue loop: 4
Continue loop: 5
```

Code Explanation:
In the `break` loop, when `i` becomes 3, the loop dies. 3, 4, and 5 are never printed. In the `continue` loop, when `i` becomes 3, only that specific loop cycle is skipped, allowing 4 and 5 to print.

> ***Real-world Example:***
> *`break`: Searching a database for a user; once found, break the loop to save time.*
> *`continue`: Processing a list of bank transactions; if one is invalid, skip it and continue processing the rest.*

> **Common Mistakes:**
> - Using `break` inside an `if` statement when you meant to break a loop. Note: `break` only breaks out of loops (and switches), it does not break out of standalone `if` blocks.

> **Best Practices:**
> - Minimize use of heavily nested jumps, as they create 'spaghetti code' which is very hard to read and debug.

Practice Task:
Write a loop from 1 to 20. Use continue to skip all even numbers, printing only odd numbers.

Note: We will discuss the `return` statement deeply in the next section on Methods.

---

5. Methods in Java

As programs grow larger, writing all logic inside `public static void main` becomes impossible to manage. Methods (also known as Functions) allow you to break your code into small, reusable, isolated blocks.

Method Calling Process

Caller (main method)

```
int x = 10;
int y = 5;
int result =
addNumbers(x, y);
```

↔

Receiver (custom method)

```
public static int
addNumbers(int a, int b) {
  return a + b;
}
```

IT STUDY HUB

Concept: Method Declaration & Calling

> **Simple Explanation:**
> A method is like a recipe. Writing the recipe down (Declaration) doesn't bake a cake. You must actually say 'Bake the cake now' (Calling) for the code to run. We do this to obey the D.R.Y. principle (Don't Repeat Yourself).

Syntax / Structure:

```
// 1. Declaration
public static returnType methodName(Parameters) {
    // code body
    return value; // (if return type is not void)
}

// 2. Calling
methodName(arguments);
```

Example Code:

```
public class MethodBasics {

    // 1. Declare the custom method (outside main!)
    public static void printGreeting() {
        System.out.println("Welcome to IT Study Hub!");
    }

    // Main is just a special method where Java starts
    public static void main(String[] args) {
        System.out.println("Starting program...");

        // 2. Call the method twice
        printGreeting();
        printGreeting();
    }
}
```

Expected Output:

```
Starting program...
Welcome to IT Study Hub!
Welcome to IT Study Hub!
```

Code Explanation:
We declared `printGreeting` OUTSIDE the `main` method, but inside the class. The `void` keyword means this method just performs an action and does not send any data back. We called it twice, saving us from writing the print statement twice.

Real-world Example:
A `calculateTax()` method can be written once and called thousands of times for every employee in a payroll system.

Common Mistakes:

- Trying to declare a custom method INSIDE the `main` method. Methods must be siblings, not nested within each other.

Best Practices:

- Method names should always be verbs in camelCase (e.g., `calculateTotal`, `printReport`, `fetchData`).

Practice Task:
Write a void method called `printDivider` that prints '----------'. Call it three times from main.

Concept: Parameters, Arguments and Return Types

> **Simple Explanation:**
> Think of a juice blender. The parameters are the fruit you put in (Apples). The method does the blending. The return value is the juice that comes out. The `return` keyword shoots the value back to whoever pressed the blend button.

Syntax / Structure:

```
public static int multiply(int a, int b) {
    return a * b;
}
```

Example Code:

```
public class ParameterReturn {

    // Accepts two ints, returns an int
    public static int getArea(int length, int width) {
        int area = length * width;
        return area; // Sends data back!
    }

    public static void main(String[] args) {
        // Calling method with arguments (5 and 10)
        int roomArea = getArea(5, 10);

        System.out.println("The area is: " + roomArea);
    }
}
```

Expected Output:

```
The area is: 50
```

Code Explanation:
`getArea` requires two inputs (parameters: `length` and `width`). In `main`, we call it and provide the actual values (arguments: `5` and `10`). The method calculates 50, and the `return` statement hands the 50 back to `main`, which gets stored in `roomArea`.

Real-world Example:
An ATM `withdraw(int amount)` method checks your balance, deducts the amount, and `returns` true (success) or false (fail).

Common Mistakes:

- Writing `return` in a method declared as `void`.
- Forgetting to store or print the returned value in the `main` method (e.g., just writing `getArea(5, 10);` does nothing visible!).

Best Practices:

- Keep methods focused on doing exactly one thing. A method shouldn't calculate the area AND print it AND save it to a database.

Practice Task:
Create a method `isAdult(int age)` that returns a boolean. Call it from main and print the result.

Parameters vs. Arguments

Aspect	Parameters	Arguments
Definition	The variables defined in the method signature. (The placeholders).	The actual values passed into the method when it is called.
Location	In the method declaration. `public void doMath(int x, int y)`	In the method call. `doMath(10, 5);`

---

6. Advanced Method Concepts

IT STUDY HUB

Concept: Method Overloading

> **Simple Explanation:**
> Java is smart. If you have an `add(int, int)` method and an `add(double, double)` method, Java looks at the data you provide when you call it, and automatically picks the correct version of the method. This is Compile-Time Polymorphism.

Syntax / Structure:

```
public static void doWork(int a) { ... }
public static void doWork(int a, int b) { ... }
public static void doWork(double a) { ... }
```

Example Code:

```
public class OverloadExample {

    // Version 1: Two ints
    public static int add(int a, int b) {
        return a + b;
    }

    // Version 2: Three ints
    public static int add(int a, int b, int c) {
        return a + b + c;
    }

    public static void main(String[] args) {
        System.out.println("Two ints: " + add(10, 20));
        System.out.println("Three ints: " + add(10, 20, 30));
    }
}
```

Expected Output:

```
Two ints: 30
Three ints: 60
```

Code Explanation:

We didn't need to invent names like `addTwoNumbers` and `addThreeNumbers`. We just

used `add` for both. Java determines which one to trigger based on the number and type of arguments passed.

Real-world Example:
`System.out.println()` is massively overloaded! You can pass it an `int`, a `String`, a `boolean`, or a `double`—it has different underlying methods for every data type.

Common Mistakes:

- Trying to overload methods by changing ONLY the return type. (e.g., `int doMath(int x)` and `double doMath(int x)`). Java will throw an error because the inputs look identical to the compiler.

Best Practices:

- Use overloading to keep your API clean and intuitive for other developers to use.

Practice Task:
Create an overloaded method `multiply()` that handles two ints, and another that handles two doubles.

Concept: Static Methods (Brief Intro)

Simple Explanation:
Notice how we always write `public static void main`? The `static` keyword means 'you don't need to build a custom Object to use this tool'. It's available globally directly from the blueprint (Class).

Syntax / Structure:

```
public static void myMethod() { ... }
```

Real-world Example:
The `Math` class in Java. You don't need to create a new Math object to calculate a square root. You just call `Math.sqrt(16)` directly because `sqrt` is a static method.

Common Mistakes:

- Trying to call a non-static method from inside `public static void main`. You will get a 'non-static method cannot be referenced from a static context' error.

Best Practices:

- For procedural/utility programs like our calculators, make all helper methods `static`. We will explore non-static Object-Oriented methods in Module 5.

7. Recursion and Variable Scope

IT STUDY HUB

Concept: Recursion

> **Simple Explanation:**
> Imagine a line of people. You ask the first person 'How many people are behind you?'. They don't know, so they ask the person behind them, and so on. The last person says 'Zero'. That answer is passed back up the chain. Recursion ALWAYS requires a 'Base Case' to stop it from calling itself forever.

Syntax / Structure:

```
public int method(int n) {
    if (baseCase) return value;
    return method(n - 1); // Calls itself!
}
```

Example Code:

```
public class RecursionExample {

    // Calculates Factorial: e.g., 5! = 5 * 4 * 3 * 2 * 1
    public static int factorial(int n) {
        // Base Case (Stops the loop)
        if (n == 1) {
            return 1;
        }
        // Recursive Call
        return n * factorial(n - 1);
    }

    public static void main(String[] args) {
        int result = factorial(4);
        System.out.println("Factorial of 4 is: " + result);
    }
}
```

Expected Output:

```
Factorial of 4 is: 24
```

Code Explanation:

`factorial(4)` calculates `4 * factorial(3)`. But it has to pause and calculate `factorial(3)`, which

is `3 * factorial(2)`, and so on until it hits `1`. Then all the answers bubble back up and multiply together.

Real-world Example:
Navigating folder directories on your computer (a folder inside a folder inside a folder).

Common Mistakes:

- Forgetting the Base Case! If you forget it, the method calls itself infinitely until the JVM throws a `StackOverflowError` and crashes.

Best Practices:

- Only use recursion for problems that naturally fit a tree or nested structure. For simple repetitions, loops are much faster and use less memory.

Practice Task:
Use recursion to print numbers counting down from 5 to 1.

Concept: Variable Scope

> **Simple Explanation:**
> Variables are trapped inside the curly braces `{}` where they are created. What happens in Vegas, stays in Vegas. What happens in a method, stays in the method.

Syntax / Structure:

```
public void myMethod() {
    int x = 10; // x only exists here
}
```

Example Code:

```
public class Scope {
    public static void main(String[] args) {
        int outside = 50;

        if (true) {
            int inside = 100;
            System.out.println(outside); // Allowed!
        }

        // System.out.println(inside); // ERROR! 'inside' is dead!
    }
}
```

Expected Output:

```
50
```

Code Explanation:
The variable `inside` was created inside the `if` block's curly braces. The moment the `if` block closes, the variable is completely destroyed in memory. `outside` was created in `main`, so it survives the entire main method.

> ***Real-world Example:***
> *Security and memory management. We don't want temporary loop counters taking up memory for the entire life of the program.*

Common Mistakes:

- Trying to access a method's parameter from inside a different method.

Best Practices:

- Keep variables as closely scoped as possible. Declare them right before you need them, not all at the top of the file.

Practice Task:
Declare a variable inside a for-loop and try to print it outside the loop to observe the compiler error.

IT STUDY HUB

8. Project: Menu-Driven Calculator

Let's fuse Control Flow and Methods. We will build an interactive calculator that runs in a continuous loop until the user explicitly chooses to exit. We will modularize the logic by putting the math operations into their own separate methods.

```
import java.util.Scanner;

public class MenuCalculator {

    // 1. HELPER METHODS (Modularization)
    public static double add(double a, double b) { return a + b; }
    public static double subtract(double a, double b) { return a - b; }
    public static double multiply(double a, double b) { return a * b; }
    public static double divide(double a, double b) {
        if (b == 0) { // Validation logic
            System.out.println("Error: Cannot divide by zero.");
            return 0;
        }
        return a / b;
    }

    // 2. MAIN METHOD
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        boolean isRunning = true; // Control flag for the loop

        // 3. CONTINUOUS LOOP
        System.out.println("=== IT STUDY HUB CALCULATOR ===");
        while (isRunning) {
            System.out.println("\nOptions: [1] Add [2] Subtract [3] Multiply [4] Divide
            System.out.print("Choose an option: ");
            int choice = scanner.nextInt();

            // 4. JUMP STATEMENT (Exit condition)
            if (choice == 0) {
                System.out.println("Shutting down... Goodbye!");
                break; // Terminates the while loop instantly
            }
```

```
            // Validation for invalid choices
            if (choice < 1 || choice > 4) {
                System.out.println("Invalid option. Try again.");
                continue; // Skips to the next loop iteration
            }

            System.out.print("Enter first number: ");
            double num1 = scanner.nextDouble();
            System.out.print("Enter second number: ");
            double num2 = scanner.nextDouble();

            double result = 0;

            // 5. DECISION LOGIC
            switch (choice) {
                case 1: result = add(num1, num2); break;
                case 2: result = subtract(num1, num2); break;
                case 3: result = multiply(num1, num2); break;
                case 4: result = divide(num1, num2); break;
            }

            System.out.println("RESULT: " + result);
        }

        scanner.close();
    }
}
```

```
=== IT STUDY HUB CALCULATOR ===

Options: [1] Add [2] Subtract [3] Multiply [4] Divide [0] Exit
Choose an option: 1
Enter first number: 10
Enter second number: 5.5
RESULT: 15.5

Options: [1] Add [2] Subtract [3] Multiply [4] Divide [0] Exit
Choose an option: 8
Invalid option. Try again.

Options: [1] Add [2] Subtract [3] Multiply [4] Divide [0] Exit
```

```
Choose an option: 0
Shutting down... Goodbye!
```

Step-by-Step Execution:

1. **Helper Methods:** We isolate the math logic into static methods at the top. This keeps `main()` clean.
2. **Control Flag:** We create a `boolean isRunning = true` to keep the `while` loop running infinitely until we manually stop it.
3. **break and continue:** If the user enters 0, we use `break` to shatter the loop and end the program. If they enter 8, we print an error and use `continue` to jump back to the top of the menu without asking for numbers.
4. **Switch statement:** Based on the valid input (1-4), the `switch` routes the data to the correct helper method.

IT STUDY HUB

9. Module Assessment and Interview Prep

Test your mastery of control flow and methods to prepare for academia and industry interviews.

A. Practice Questions (Test Your Logic)

1. Write an `if-else` statement to determine if a given year (e.g., 2024) is a leap year.
2. Refactor a 5-step `else-if` ladder that checks an integer 1-5 into a `switch` statement.
3. Write a `for` loop that calculates the sum of all numbers from 1 to 50.
4. Write a nested `for` loop that prints a 5x5 grid of `#` symbols.
5. What is the difference in behavior if you replace a `break` statement with a `continue` statement inside a loop?
6. Declare a static method called `greetUser` that takes a `String name` as a parameter and returns a concatenated greeting.
7. Write an overloaded method `getArea`. One version takes a single `double` (radius of circle), the other takes two `double`s (length and width of rectangle).
8. Identify the error: `public void test() { int x = 5; } public void print() { System.out.println(x); }`
9. Write a recursive method that counts down from `n` to 1, printing each number, and prints 'Blastoff!' at 0.
10. Create a `do-while` loop that forces a user to enter a positive number. If they enter a negative number, it asks again.
11. What will be the output of a `switch` statement if the `break` is omitted on case 1, and the input matches case 1?
12. Explain why a `while` loop might execute 0 times, but a `do-while` loop will always execute at least once.
13. Write a Java program to find the largest of 3 numbers using nested `if` statements.
14. What is variable shadowing? (Research hint: declaring a variable inside a block with the same name as a class variable).
15. Trace the execution of `factorial(3)` on paper. Write down exactly what each recursive step returns.

IT STUDY HUB

B. University Viva Questions & Answers

Q: What are Control Flow statements?

Ans: Statements that alter the sequential execution of a program based on conditions or loops (e.g., if, switch, for, while).

Q: What is the difference between if-else and switch?

Ans: if-else evaluates boolean conditions (true/false) and ranges. switch evaluates specific fixed values for equality and is cleaner for multiple discrete choices.

Q: Why do we use the 'break' keyword in a switch statement?

Ans: To prevent 'fallthrough'. Without break, the JVM will execute all subsequent cases after a match is found until the end of the switch block.

Q: What are the three components of a for loop declaration?

Ans: Initialization (setting the counter), Condition (evaluating when to stop), and Increment/Decrement (updating the counter).

Q: When should you use a while loop instead of a for loop?

Ans: When the number of iterations is unknown in advance and depends entirely on a runtime condition (like waiting for specific user input).

Q: What is the primary difference between while and do-while?

Ans: The while loop checks the condition before executing the body (Pre-test). The do-while loop executes the body first, then checks the condition (Post-test), guaranteeing at least one execution.

Q: What is an infinite loop?

Ans: A loop whose terminating condition is never met, causing it to run endlessly until the program crashes or is manually killed.

Q: What does the 'continue' keyword do?

Ans: It immediately stops the current iteration of a loop and jumps back to the loop's condition/update step to begin the next iteration.

Q: What is a Method?

Ans: A named block of reusable code designed to perform a specific task. It executes only when called.

IT STUDY HUB

Q: What does the 'void' keyword mean?

Ans: It indicates that the method will not return any data to the caller upon completion.

Q: What is the difference between an Argument and a Parameter?

Ans: Parameters are the variables defined in the method declaration. Arguments are the actual values passed into the method when it is called.

Q: What is Method Overloading?

Ans: Defining multiple methods in the same class with the exact same name, but with a different number or type of parameters.

Q: Can we overload a method by only changing its return type?

Ans: No. Method overloading is determined strictly by the parameter signature. Changing only the return type causes a compile-time error.

Q: What is the 'return' keyword used for?

Ans: To exit a method and optionally pass a specific value back to the caller.

Q: What is Recursion?

Ans: A programming technique where a method calls itself in order to solve a smaller instance of the same problem.

Q: What is a Base Case in recursion?

Ans: The condition under which a recursive method stops calling itself and begins returning. Without it, recursion leads to a StackOverflowError.

Q: What is Variable Scope?

Ans: The region of the code where a variable is accessible. In Java, scope is strictly defined by curly braces {}.

Q: What happens if you define a variable inside an if-block?

Ans: It becomes a block-scoped variable. It is created when the block starts and destroyed when the block ends; it cannot be accessed outside the block.

Q: What is a static method?

Ans: A method that belongs to the Class itself rather than an instance (object) of the class. It can be called without creating an object.

Q: Can a static method call a non-static method directly?

Ans: No, a static method (like main) cannot directly access non-static methods or variables without first creating an instance of the class.

IT STUDY HUB

C. Beginner Interview Questions

Interview Q: Explain the concept of 'Fallthrough' in a switch statement.

Ans: Fallthrough occurs when a case matches, but there is no 'break' statement at the end of its code block. The JVM will completely ignore subsequent case labels and continuously execute all code downwards until it hits a break or the end of the switch block. This is sometimes used intentionally to group cases together.

Interview Q: Is it possible to use Strings in a switch statement?

Ans: Yes, starting from Java 7, Strings are fully supported in switch statements. However, the comparison is case-sensitive, so 'apple' and 'Apple' will evaluate as different cases.

Interview Q: How would you break out of a nested loop completely from the inner loop?

Ans: By using a 'Labeled Break'. You place a label (e.g., 'outerLoop:') above the first loop, and inside the nested loop, use 'break outerLoop;'. A standard 'break' will only break out of the innermost loop.

Interview Q: Explain why recursion can be dangerous for memory.

Ans: Every time a method is called, Java allocates a new frame on the Call Stack to store its local variables. In recursion, if the base case is missing or takes too long to reach, thousands of frames pile up, consuming all stack memory and throwing a fatal StackOverflowError.

Interview Q: Why is method overloading considered Compile-Time Polymorphism?

Ans: Because the Java compiler determines exactly which version of the overloaded method to call during the compilation phase, based on the data types of the arguments provided. It doesn't wait until runtime to figure it out.

Interview Q: If you have an if-else ladder with 50 conditions, why might a switch statement be better?

Ans: A switch statement is generally more readable. Furthermore, modern JVM compilers optimize switch statements using lookup tables (jump tables), making execution mathematically faster than evaluating 50 sequential boolean if-else conditions.

Interview Q: Can a method return multiple values in Java?

Ans: No, a method can strictly return only one primitive value or one object reference. To return multiple values, you must bundle them into an Array, a Collection, or a custom Class object and return that single container.

IT STUDY HUB

Interview Q: What is the unreachable code error?

Ans: It is a compile-time error thrown when you place code immediately after a 'return', 'break', or 'continue' statement within the same block, or after an infinite loop. The compiler knows that code can mathematically never execute.

Interview Q: What are the advantages of modularizing code into methods?

Ans: Code Reusability (DRY principle), Readability (abstracting complex logic behind a simple name), and Maintainability (fixing a bug in one method automatically fixes it everywhere that method is called).

Interview Q: How does Variable Scope prevent memory leaks in loops?

Ans: Variables declared inside a loop block are instantiated and destroyed on every single iteration. Once the loop finishes, those variables are fully wiped from local scope memory by the Garbage Collector, keeping the application lightweight.

IT STUDY HUB

10. Module Summary and Key Terms

Key Terms Glossary

- **Control Flow:** The order in which statements are executed in a script.
- **Iteration:** A single execution of a looped block of code.
- **Fallthrough:** When a switch statement executes subsequent cases due to a missing break.
- **Method Signature:** The combination of the method's name and its parameter list (e.g., `add(int, int)`).
- **Scope:** The strictly bounded area (defined by `{}`) where a variable exists.
- **DRY Principle:** "Don't Repeat Yourself". A coding philosophy achieved by using methods and loops.

Module Summary

In this module, you took control of your program's destiny. You learned how to inject intelligence using `if`, `else-if`, and `switch` logic, allowing your applications to react to varying conditions and user inputs.

You mastered efficiency by replacing thousands of repetitive lines of code with compact `for`, `while`, and `do-while` loops, utilizing `break` and `continue` for precise traversal. Furthermore, you leveled up your software engineering skills by modularizing logic into reusable **Methods**, mastering the data transfer bridge of parameters and return types.

Finally, we brought all these concepts together to build a robust, continuous, menu-driven calculator.

WHAT COMES NEXT?

Module 4 — Arrays, Strings & Basic Data Handling

Currently, if you want to store the grades of 100 students, you have to create 100 separate variables. That is highly inefficient! In the next volume, you will master Arrays—a way to store lists of data in a single variable. You will also dive deep into the String class to manipulate text, search for words, and prepare for real-world data processing.

IT STUDY HUB SERIES